

中山大学专业课《程序设计I》

第九节 字符串 (下)

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

字符串的表示方式

字符串的三种表示方式

1. 字符串常量（字面量）

"China"

...	'C'	'h'	'i'	'n'	'a'	'\0'	...
-----	-----	-----	-----	-----	-----	------	-----

2. 字符数组

char s[8]="China"

	s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	
...	'C'	'h'	'i'	'n'	'a'	'\0'	'\0'	'\0'	...

3. 字符指针

char *p2=s

	p2		s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	
...	1010	...	'C'	'h'	'i'	'n'	'a'	'\0'	'\0'	'\0'	...
1004		1010	1011	1012	1013	1014	1015	1016	1017		

const char *p1="China"

	p1									
...	2000	...	'C'	'h'	'i'	'n'	'a'	'\0'	...	
1000		2000	2001	2002	2003	2004	2005			

字符串常量的值是字符串首字符的地址

以下代码会输出什么结果?

```
printf("%s %p %c\n", "We", "are", *"space farers.");
```

字符串的类型

事实上，字符串就是以空字符（'\0'）结尾的char类型数组

1. 字符数组（`char s[8]="China"`）是 `char[]`
2. 字符串常量（`"China"`）是 `const char[]`
3. 字符指针（`const char *p1="China"`）是 `const char *`

因此，`s`、`"China"`、`p1`的值都是字符串首字符的地址

字符串的输入输出

常用字符串输入输出函数

常用输入函数

1. `scanf("%s", str)`
2. `gets(str)`
3. `fgets(str, n, stdin)`

常用输出函数

1. `printf("%s", str)`
2. `puts(str)`
3. `fputs(str, stdout)`

常用输出函数

`char str[8]="China"`

...	'C'	'h'	'i'	'n'	'a'	'\0'	'\0'	'\0'	...
-----	-----	-----	-----	-----	-----	------	------	------	-----

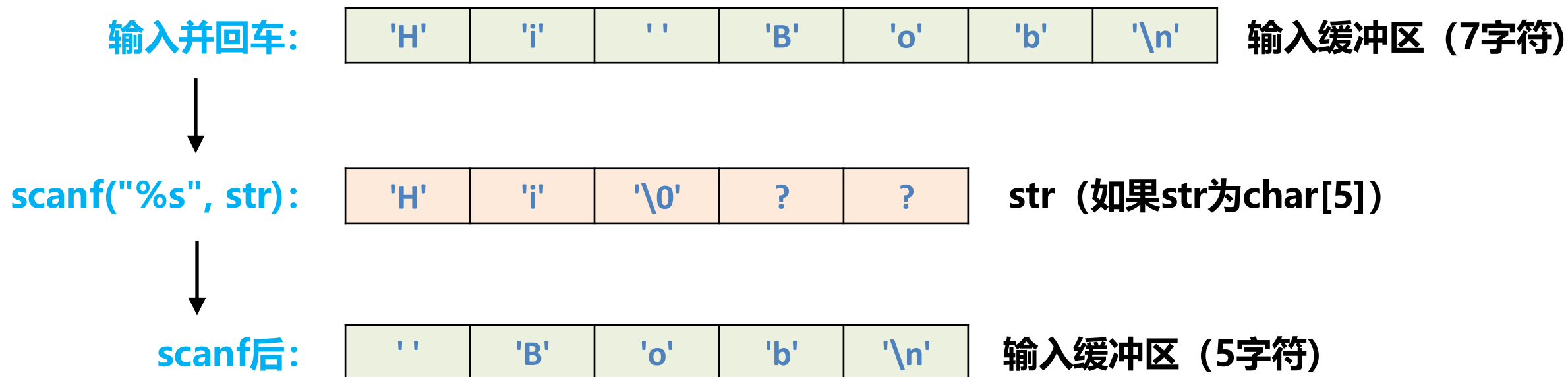
1. `printf("%s", str)`
2. `fputs(str, stdout)`
3. `puts(str)`

相同点：都是从`str`指向的地址开始，逐字节输出字符，直到遇到第一个空字符为止

不同点：`puts(str)`会在字符串末尾加换行符（`'\n'`）

使用 scanf 读入字符串

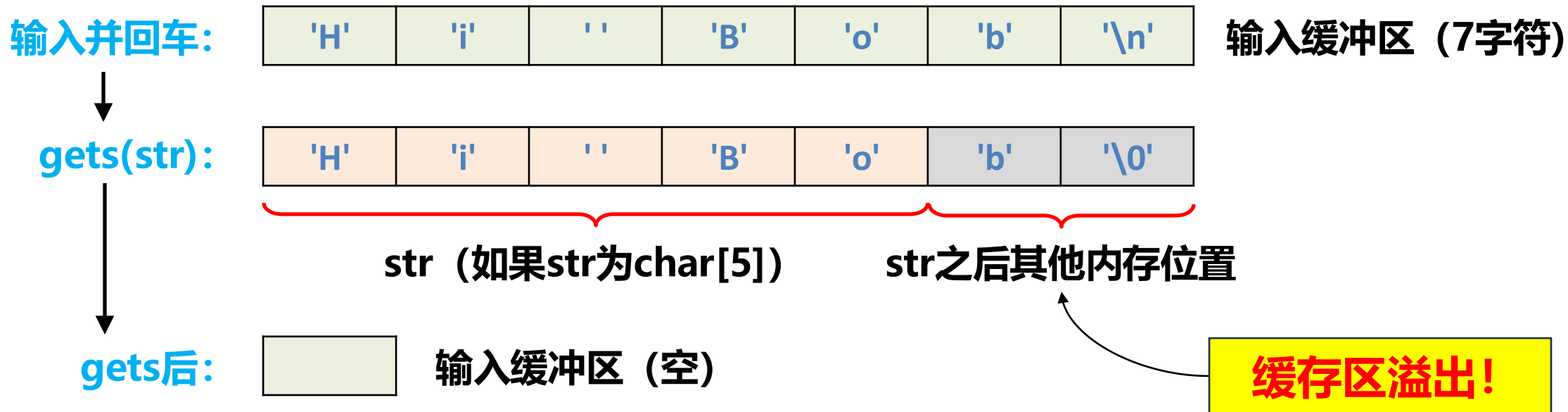
键盘输入字符串 **“Hi Bob”** 并按下 **“回车键”** 后，键盘的**输入（包含换行符）** 被放到C程序的**输入缓冲区**中，随后字符串输入函数会从输入缓冲区中读取字符



scanf("%s", str)遇到**结尾空白字符**会停止读入，并在字符串**末尾加'\0'**

使用 gets 读入字符串（不安全）

键盘输入字符串“Hi Bob”并按下“回车键”后，键盘的输入（包含换行符）被放到C程序的输入缓冲区中，随后字符串输入函数会从输入缓冲区中读取字符

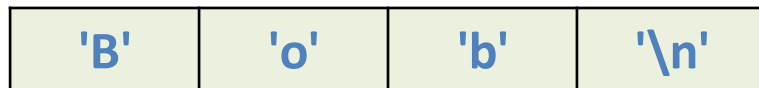


gets()读取整行字符，直至遇到换行符，丢弃换行符，并在字符串末尾加'\0'

使用 gets 读入字符串（不安全）

键盘输入字符串 “Bob” 并按下 “回车键” 后.....

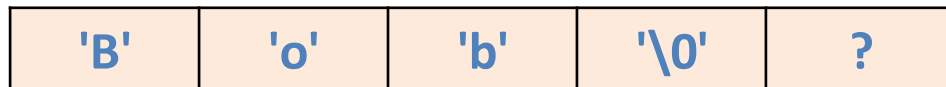
输入并回车:



输入缓冲区 (4字符)



gets(str):



str (如果str为char[5])



gets后:

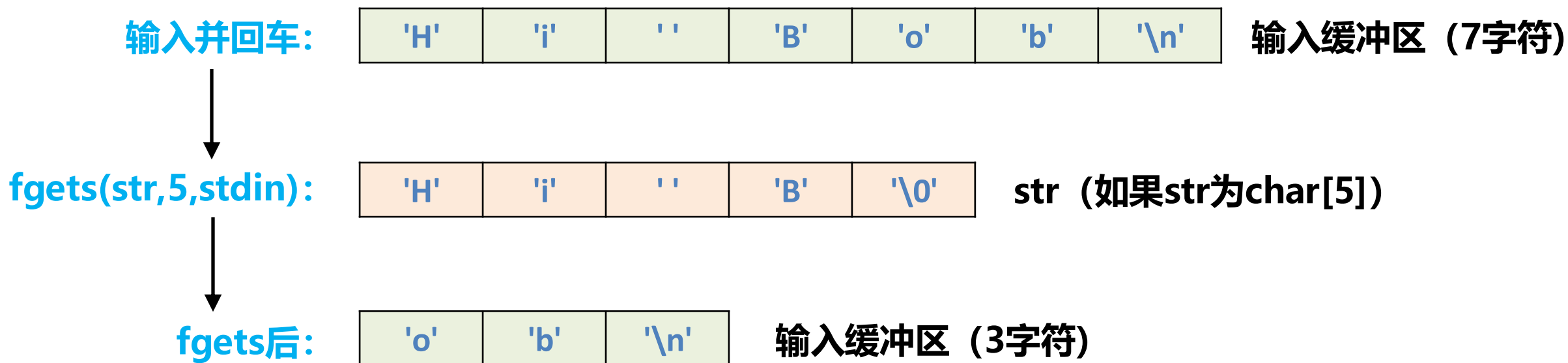


输入缓冲区 (空)

gets()读取整行字符，直至遇到换行符，丢弃换行符，并在字符串末尾加'\0'

使用 fgets 读入字符串（更安全的选择）

键盘输入字符串 “Hi Bob” 并按下 “回车键” 后，键盘的输入（包含换行符）被放到C程序的输入缓冲区中，随后字符串输入函数会从输入缓冲区中读取字符



fgets(str,n,stdin)读入n-1个字符或遇到第一个换行符为止，并在字符串末尾加'\0'

注意：若读到了换行符，fgets会把'\n'放入字符串中

使用 fgets 读入字符串（更安全的选择）

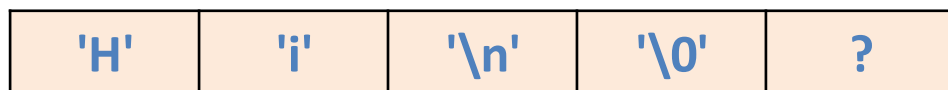
键盘输入字符串 **"Hi"** 并按下 **"回车键"** 后.....

输入并回车:



输入缓冲区 (3字符)

fgets(str,5,stdin):



str (如果str为char[5])

fgets后:



输入缓冲区 (空)

fgets(str,n,stdin)读入**n-1**个字符或遇到**第一个换行符**为止，并在字符串**末尾加'\0'**

注意：若读到了换行符，fgets会把**'\n'**放入字符串中

字符串函数

常用字符串函数（声明在头文件string.h中）

功能	声明
算字符串长度	<code>size_t strlen(const char *s);</code>
连接字符串	<code>char *strcat(char *s1, const char *s2);</code> <code>char *strncat(char *s1, const char *s2, size_t n);</code>
拷贝字符串	<code>char *strcpy(char *s1, const char *s2);</code> <code>char *strncpy(char *s1, const char *s2, size_t n);</code>
比较字符串	<code>int strcmp(const char *s1, const char *s2);</code> <code>int strncmp(const char *s1, const char *s2, size_t n);</code>
检索字符（串）	<code>char *strchr(const char *s, int c);</code> <code>char *strrchr(const char *s, int c);</code> <code>char *strstr(const char *s1, const char *s2);</code>

返回字符串长度：strlen(str)

从str指向的地址开始，逐字节计数，直到遇到**第一个空字符** ('\0') 为止

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char str[] = "Hello!";
    unsigned int l1,l2;

    l1= strlen(str);
    l2= strlen("C PROGRAMMING!");
    printf("l1 = %u, l2 = %u\n\n", l1, l2);
}
```

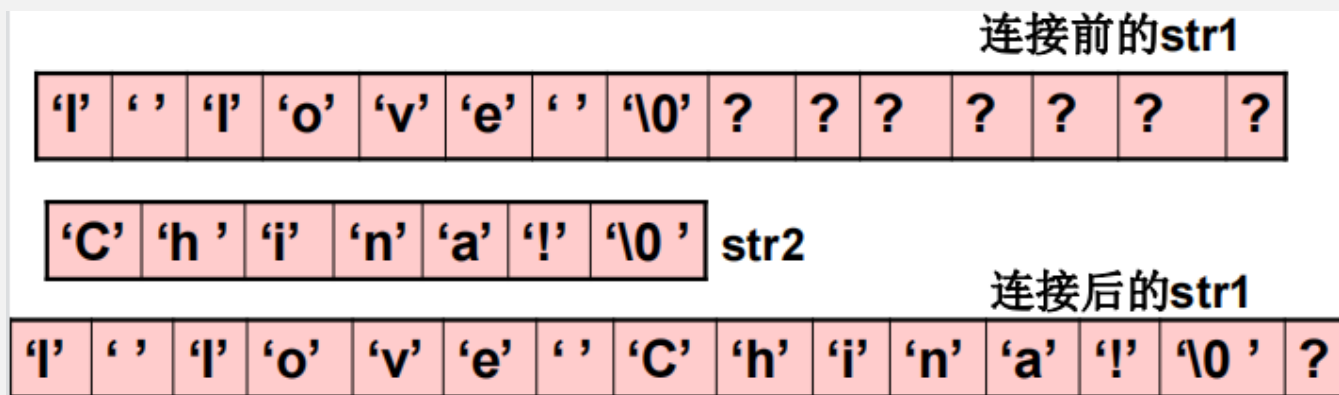
l1 = 6, l2 = 14

字符串连接: `strcat(str1, str2)`

- (1) 去掉str1的结束符, 并将str2指向的字符串拷贝到str1末尾
- (2) **str1数组必须够长**, 否则产生不可知错误 (缓冲区溢出)
- (3) 返回**str1数组的首地址**, 即str1

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[15] = {"I love "};
    char str2[] = {"China!"};
    printf("%s", strcat(str1, str2));
}
```

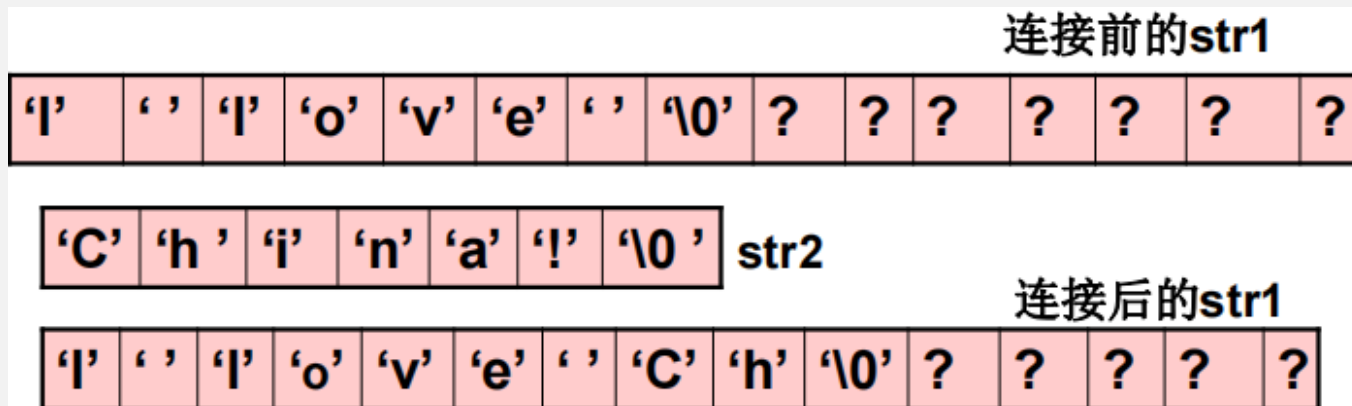


字符串连接: `strncat(str1, str2, n)`

把`str2`指向字符串的前`n`个字符连接到`str1`上, 并在结尾添加`'\0'`

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[15] = {"I love "};
    char str2[] = {"China!"};
    printf("%s", strncat(str1, str2, 2));
}
```



不会拷贝`str2`空字符和其后的字符, `str1`中需保证有不少于`n`字节的空间

字符串拷贝：strcpy(str1, str2)

将str2指向的字符串拷贝到str1指向的位置，返回str1

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[10] = {};
    strcpy(str1, "China");
}
```

执行后，str1：

C	h	i	n	a	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

str1必须定义得足够大，以容纳被复制的str2

字符串拷贝：strncpy(str1, str2, n)

将str2指向字符串的前n个字符拷贝到str1指向的位置，返回str1

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[10];
    char str2[] = "China";
    strncpy(str1, str2, 2);
}
```

执行后，str1：

C	h	?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---	---	---

注意：拷贝时不会在str2的前n个字符后单独加'\0'

字符串比较: strcmp(str1, str2)

比较str1和str2 (**不能**直接比较**str1 == str2**)

```
#include<string.h>
#include<stdio.h>
int main(void) {
    char str1[] = "Hell";
    char str2[] = "Hello";
    printf("%d\n", strcmp(str1, str2));
    strcpy(str1, "ABcD");
    strcpy(str2, "ABCD");
    printf("%d\n", strcmp(str1, str2) );
    printf("%d\n", strcmp("fine", "fine"));
}
```



-1
1
0
0

str1和str2按照**字符在字符集中的表示值**进行比较: 从第一个字符开始, 逐个字符两两进行比较; 第一对不同的字符出现时, 较大字符所在的字符串就是大者; 如果其中较短的字符串**是较长字符串前面的子字符串**, 则较长的字符串大于较短的字符串。

函数返回值:

str1 **小于** str2时, 返回**小于0**的整数值

str1 **等于** str2时, 返回整数值**0**

str1 **大于** str2时, 返回**大于0**的整数值

字符串比较: `strncmp(str1, str2)`

比较str1与str2头n个字符的大小，返回值与strcmp相同

字符检索: `strchr(str, c)`

`char * strchr(const char *str, int c)`

返回`str`所指向的字符串中**首次出现字符`c`**（包括空字符）位置的**指针**

若未找到，返回**空指针**

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str[] = "abcdedcbaf";
    printf("%d\n", strchr(str, 'd') - str); // 'd'在str中第一次出现的下标
    printf("%s\n", strchr(str, 'd'));      // 从第一次'd'出现开始的字符串
}
```



3
abcdedcbaf

字符检索: `strrchr(str, c)`

`char *strrchr(const char *str, int c)`

返回`str`所指向的字符串中**最后一次出现字符`c`**（包括空字符）位置的**指针**
若未找到，返回**空指针**

```
#include <string.h>
#include <stdio.h>
```

```
int main(void) {
    char str[] = "abcdedcbaf";
    printf("%d\n", strrchr(str, 'd') - str); // 'd'在str中第一次出现的下标
    printf("%s\n", strrchr(str, 'd'));       // 从第一次'd'出现开始的字符串
}
```

5
dcbaf

字符串检索: strstr(str1, str2)

`char *strstr(const char *str1, const char *str2)`

返回字符串str1中第一次出现字符串str2位置的指针 (不含终止符'\0')

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str[] = "abcdedcbaf";
    printf("%d\n", strstr(str, "ded") - str);
    // "ded"在str中第一次出现的下标
    printf("%s\n", strstr(str, "ded"));
    // 从第一次"ded"出现开始的字符串
}
```



```
3
dedcbaf
```

大小写转换（非标准库函数）： `strlwr(str)`, `strupr(str)`

`strlwr(str)`：将`str`指向的字符串字母全部转化为小写

`strupr(str)`：将`str`指向的字符串字母全部转化为大写

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[] = "Hello";
    char ch = 'a';
    strlwr(str1); puts(str1);
    strupr(str1); puts(str1);
    strupr(&ch);
    printf("%c\n\n", ch);
}
```



hello
HELLO
A

常用字符串函数（声明在头文件string.h中）

功能	声明
字符串长度	<code>size_t strlen(const char *s);</code>
字符串连接	<code>char *strcat(char *s1, const char *s2);</code> <code>char *strncat(char *s1, const char *s2, size_t n);</code>
字符串拷贝	<code>char *strcpy(char *s1, const char *s2);</code> <code>char *strncpy(char *s1, const char *s2, size_t n);</code>
字符串比较	<code>int strcmp(const char *s1, const char *s2);</code> <code>int strncmp(const char *s1, const char *s2, size_t n);</code>
字符（串）检索	<code>char *strchr(const char *s, int c);</code> <code>char *strrchr(const char *s, int c);</code> <code>char *strstr(const char *s1, const char *s2);</code>

谢谢！

邮箱: yuesh5@mail.sysu.edu.cn